COMP 4910 – CAPSTONE PROJECT | MILESTONE 2 GROUP PRESENTATION

Zero Trust Engine for WSN and IoT

Phase 1: RF data acquisition system for Zigbee devices

Magdala Nwuke • Kpughur-Tule Tertsegha Moses • Pranay Sethi • Ashima Bhasin

Department of Computing Science | Thompson Rivers University

Project Overview

The system

- A Raspberry Pi gateway with an ADALM-PLUTO SDR captures raw I/Q samples from known Zigbee transmitters.
- Recordings and metadata are stored locally.
- A browser dashboard controls captures and shows results.

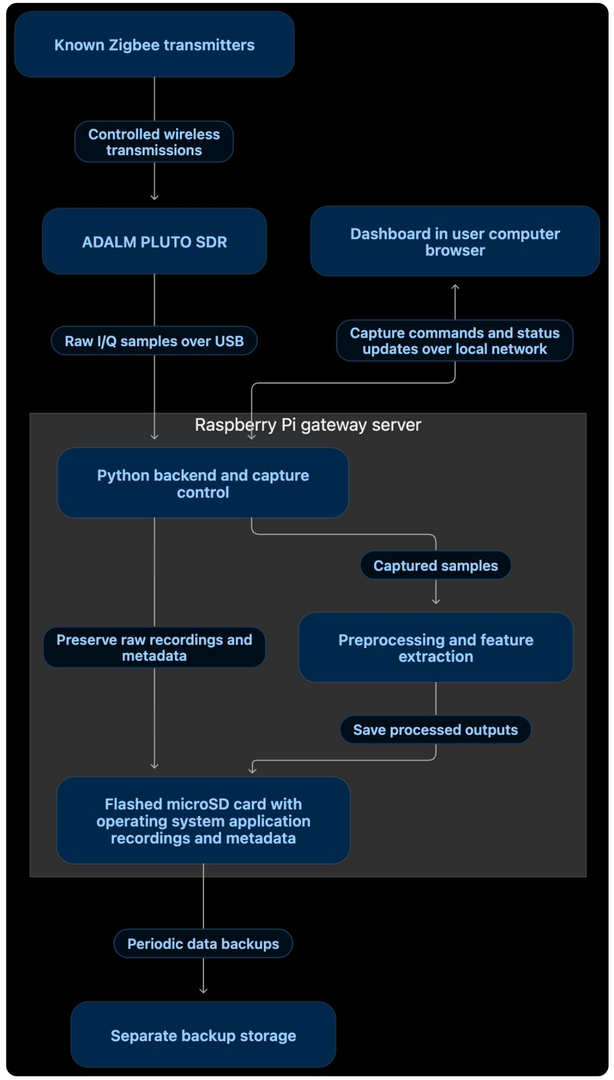
The problem

- RF fingerprinting needs a labeled, reproducible dataset of known transmitters.
- This phase builds the acquisition and storage foundation for that dataset.
- Enforcement of security is out of scope.

Main goals

- Capture and label I/Q samples from known Zigbee devices
- Support a two-to-four-day collection period
- Preserve raw recordings and metadata for repeated analysis
- Run fully locally, with no cloud dependencies

System Architecture



Major components

- Known Zigbee transmitters as labeled sources
- ADALM-PLUTO SDR captures raw I/Q
- Raspberry Pi gateway: Python backend, preprocessing, feature extraction
- microSD storage and a browser dashboard
- Separate backup storage

Communication

- SDR to Pi: raw I/Q samples over USB
- Dashboard and backend: capture commands and status over the local network
- Backend to microSD: recordings, metadata, outputs
- Periodic backups to a separate device
- No external or cloud services

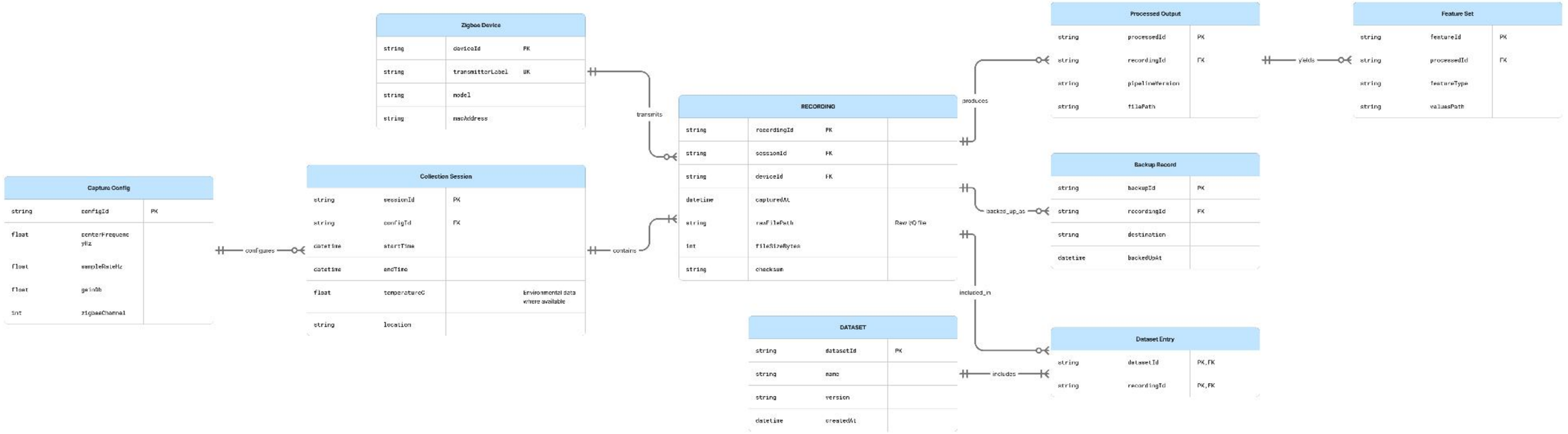
Hosting and deployment

- All services run on the Raspberry Pi gateway
- Dashboard served by the Pi, opened from a computer browser on the local network
- Fully local operation
- Scheduled acquisition, error logging, storage monitoring

Decisions tied to requirements

- Local-only: no cloud dependencies
- Raw recordings kept: repeated analysis
- Scheduled capture and backups: 2–4 day collection
- Designated transmitters: accurate labels
- Saved configurations: reproducibility

Data Model



Key entities

- Zigbee Device, Collection Session, Capture Config
- Recording: one captured I/Q file
- Processed Output and Feature Set
- Dataset, Dataset Entry, Backup Record

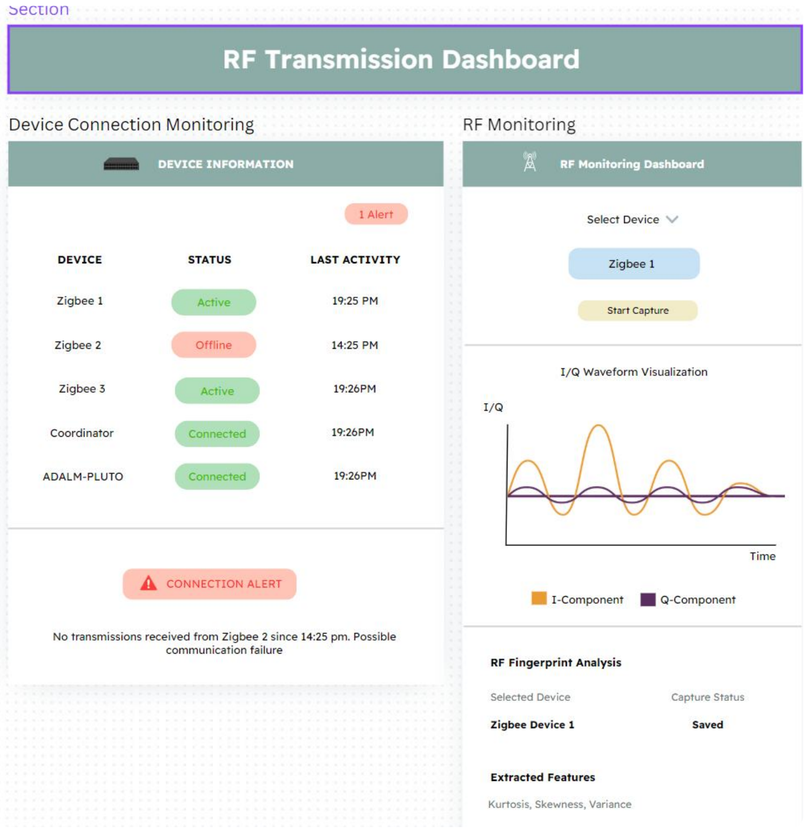
Relationships

- A device transmits many recordings
- A session contains recordings; a config configures a session
- A recording produces processed output, which yields feature sets
- Datasets include recordings via dataset entries

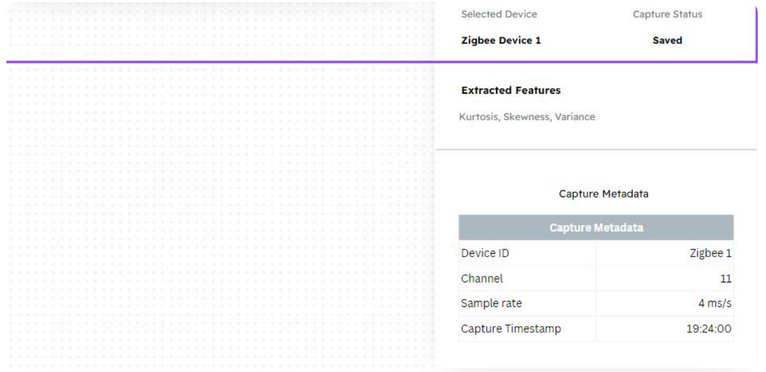
Storage and access

- Raw I/Q and processed files kept in organized directories
- Identifiers, parameters, and file paths held in a metadata repository
- Recordings linked to files by path and checksum

Interface Design



RF Data Collection Dashboard wireframe: device connection monitoring (left) and RF monitoring (right).



How users navigate

- Device Connection Monitoring: status and last activity for each Zigbee device, the coordinator, and the SDR
- Connection alerts flag devices that stop transmitting
- RF Monitoring: pick a device from the dropdown, then start a capture
- Capture Metadata table: review extracted features and parameters

Capture workflow

1. Select a Zigbee device from the dropdown
2. Click Start Capture
3. The Pi starts capture through the ADALM-PLUTO SDR
4. I/Q samples are saved and shown as a waveform
5. Features and metadata appear in the Capture Metadata table

Interactions and APIs

- Dashboard to Pi software: start captures, request device status
- Pi to dashboard: waveforms, metadata, results
- SDR to Pi: raw I/Q over USB
- Coordinator to Pi: Zigbee packet data

Requirements Traceability

Requirement	Design response	Component / entity
Capture raw I/Q from known Zigbee transmitters	ADALM-PLUTO SDR feeds the Raspberry Pi over USB	SDR, Pi gateway
Label every recording accurately	Designated transmitters and controlled transmission schedules	Zigbee Device, Recording
Record acquisition parameters	Sampling rate, frequency, channel, and gain saved per session	Capture Config, Collection Session
Monitor devices and run captures	Status panel, device selector, Start Capture, alerts	RF Dashboard
Preserve data for repeated analysis	Raw files kept unchanged; processed outputs stored separately	Recording, Processed Output, Feature Set
Support dataset reproducibility	Saved configurations, organized datasets, backups	Dataset, Dataset Entry, Backup Record

Non-functional drivers

Local-only operation (no cloud) • Reliable 2–4 day collection • Reproducibility • Storage validated against sampling rate and capture schedule

Technology Selection and Justification

Frontend

- HTML, CSS, and JavaScript
- RF Data Collection Dashboard
- Select devices, start or stop recordings, view I/Q signals
- No complex framework needed

Backend

- Python with Flask connects the dashboard to data collection
- pyadi-iio talks to the ADALM-PLUTO SDR
- NumPy and SciPy process I/Q: variance, skewness, kurtosis

Database

- SQLite: devices, sessions, timestamps, capture settings, file locations
- Raw I/Q and outputs in organized folders on the Pi
- Lightweight, no database server

Hosting and deployment

- Runs locally on Raspberry Pi OS
- Dashboard in a browser on the same local network
- microSD holds OS, app, data; backups on a separate device
- Test storage and write speed first

Why these technologies

- Work well together and are widely documented
- Suit our hardware and budget
- Python offers signal-processing libraries
- Flask and SQLite keep the system simple
- Browser access: nothing to install

Alternatives rejected

- React: adds complexity the dashboard does not need
- Node.js: Python has established scientific and SDR libraries
- MySQL / PostgreSQL: need extra setup
- AWS: cloud dependency and possible hosting costs

How this meets the requirements

- Functional: manage recordings, monitor acquisition, view signals, organize metadata
- Non-functional: affordability, local operation, maintainability, reproducibility
- Original I/Q and metadata kept for repeated analysis and future RF fingerprinting

Licence Inventory

Technology	Version	Licence	Purpose
Python	3.13	PSF	Backend programming
Flask	3.1.3	BSD-3-Clause	Dashboard communication
NumPy	2.5.4	BSD-3-Clause	I/Q data processing
SciPy	1.18.1	BSD-3-Clause	Statistical feature extraction
pyadi-iio	0.0.21	ADI BSD-style	SDR communication
libiio	0.26 or 1.0	LGPL-2.1-or-later*	Low-level SDR interface
SQLite	3.46.1	Public domain	Metadata storage
Raspberry Pi OS	Debian 13 (Trixie), 64-bit	Multiple open-source licences	Local hosting and storage

Obligations and risks

- Mostly open source with no licence fees; preserve required copyright notices when redistributing
- * libiio has extra conditions that depend on how it is distributed
- Final dependency versions and licences will be checked before deployment
- No external cloud APIs are required